Code Explanation

Source Record Count Check Code Explanation

The provided Python script implements a source record count check for reconciliation and audit purposes. It uses Apache Spark to extract data from an Oracle database, count the records, and write the result to a target flat file.

Overview of the Code

This script is designed to perform a source record count check as defined in the S2T document for mapping m_E2E_AC_TXDBH_DP_YUYU_SRC_REC_CNT_CHK. It creates a Spark session, extracts data from the source system using a provided SQL query, calculates the record count, and writes the result to a target file.

Step 1: Creating a Spark Session
The first step is to create a Spark session with the required configuration. This involves setting the application name and configuring Spark SQL properties.

```
def create_spark_session() -> SparkSession:
    return (SparkSession.builder
            .appName("E2E_AC_TXDBH_DP_YUYU_SRC_REC_CNT_CHK")
            .config("spark.sql.legacy.timeParserPolicy", "LEGACY")
            .config("spark.sql.sources.partitionOverwriteMode", "dyn
amic")
            .getOrCreate())
```

Step 2: Extracting Source Data
The next step is to extract data from the source system using the provided SQL query. This is done using the `extract_source_data` function, which takes a Spark session and the SQL query as input.

```
def extract_source_data(spark: SparkSession, m_src_sql: str) -> Data
Frame:
    try:
        df = spark.read.format("jdbc")
            .option("url", "jdbc:oracle:thin:@//oracle-host:1521/ser
vice_name")
            .option("dbtable", f"({m_src_sql})")
            .option("user", "ZSYSE2EDEV")
            .option("password", "{{secrets/oracle/password}}")
            .option("driver", "oracle.jdbc.driver.OracleDriver")
            .load()
        return df
    except Exception as e:
        logger.error(f"Error extracting source data: {str(e)}")
        raise
```

Step 3: Calculating Source Record Count
After extracting the data, the script calculates the source record count using the `calculate_source_count` function. This function takes the extracted DataFrame as input and returns a new DataFrame with the count.

```
def calculate_source_count(df: DataFrame) -> DataFrame:
    count_df = df.agg(F.count("*").alias("SOURCE_REC_COUNT"))
    count_df = count_df.withColumn("SOURCE_RECT",
                                   F.when(F.length(F.col("SOURCE_REC
_COUNT").cast("string")) <= 10,
                                          F.col("SOURCE_REC_COUNT").
cast("string"))
                                   .otherwise(F.lit("OVERFLOW")))
    return count_df.select("SOURCE_RECT")
```

Step 4: Writing to Target File
The final step is to write the calculated count to a target flat file using the `write_to_target` function. This function takes the count DataFrame, output path, and output filename as input.

```
def write_to_target(df: DataFrame, output_path: str, output_filename
: str) -> None:
    try:
        df.coalesce(1).write.mode("overwrite").format("csv")
            .option("header", "false")
            .option("delimiter", ",")
            .save(f"{output_path}/temp")

        spark = SparkSession.getActiveSession()
        temp_file = spark.read.format("text").load(f"{output_path}/t
emp").coalesce(1)
        temp_file.write.mode("overwrite").format("text").save(os.pat
h.join(output_path, output_filename))

        logger.info(f"Successfully wrote source count to {os.path.jo
in(output_path, output_filename)}")
    except Exception as e:
        logger.error(f"Error writing to target: {str(e)}")
        raise
```

Step 5: Running the Source Count Check Process
The `run_source_count_check` function is the main entry point for the script. It orchestrates the entire process by creating a Spark session, extracting source data, calculating the source count, and writing the result to a target file.

```
def run_source_count_check(
    m_src_sql: str,
    output_path: str,
    output_filename: Optional[str] = None
) -> None:
    try:
        spark = create_spark_session()
        source_df = extract_source_data(spark, m_src_sql)
        count_df = calculate_source_count(source_df)
        write_to_target(count_df, output_path, output_filename or f"
SOURCE_COUNT_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv")
        logger.info(f"Source count check completed successfully")
```

```
    except Exception as e:
        logger.error(f"Source count check failed: {str(e)}")
        raise
```

Step 6: Command Line Argument Parsing and Execution

The script uses the `argparse` library to parse command-line arguments and execute the `run_source_count_check` function with the provided inputs.

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Run source record
count check")
    parser.add_argument("--m-src-sql", required=True, help="SQL quer
y for source data extraction")
    parser.add_argument("--output-path", required=True, help="Output
 directory path")
    parser.add_argument("--output-filename", help="Output filename")

    args = parser.parse_args()

    run_source_count_check(
        m_src_sql=args.m_src_sql,
        output_path=args.output_path,
        output_filename=args.output_filename
    )
```